

Implementación y Comparación de Algoritmos de Ruta Más Corta sobre Grafos Representados mediante Matrices de Adyacencia

Samuel David Martínez Rivera – Lucía Alejandra Vega Linares

Programa de Ingeniería de Sistemas, Universidad de los Llanos, Villavicencio, Meta, Colombia

Docente: Marco Antonio Gutiérrez Cárdenas — Octubre de 2026

Resumen— Este documento presenta el desarrollo de una aplicación en Python que modela grafos dirigidos y no dirigidos, con y sin pesos, a partir de su matriz de adyacencia. El programa valida los datos de entrada, genera la representación matemática $G = (V, E)$ y la representación gráfica del grafo, y ofrece evidencias de los conceptos de nodos adyacentes, camino y ciclo. Para el cálculo de rutas más cortas se implementaron tres variantes: Dijkstra con búsqueda lineal del mínimo, Dijkstra con lista de adyacencia y cola de prioridad (heap) y Bellman-Ford, este último capaz de manejar pesos negativos y de detectar ciclos de peso negativo. La aplicación cuenta con una interfaz gráfica en Tkinter y un módulo de comparación de eficiencia. En las pruebas, las tres variantes entregaron la misma ruta y el mismo costo; Bellman-Ford mostró un crecimiento de tiempo consistente con $O(V^3)$, mientras que las variantes de Dijkstra fueron considerablemente más rápidas.

Palabras clave— teoría de grafos, matriz de adyacencia, Dijkstra, Bellman-Ford, ruta más corta, detección de ciclos, Python.

I. INTRODUCCIÓN

Encontrar la ruta más corta entre dos nodos es una tarea fundamental en los modelos de redes y tiene aplicaciones en redes de transporte, telecomunicaciones, redes sociales y optimización de procesos [1], [2]. Un grafo se define como $G = (V, A)$, donde V es el conjunto de vértices y A el conjunto de aristas, y cada arista puede tener un peso que representa un costo o una distancia.

Este trabajo integra el contenido de dos prácticas del curso de Optimización. La Práctica No. 03 pide ingresar la matriz de adyacencia de un grafo, validar los datos y generar la representación matemática, la representación gráfica y evidencias de camino, ciclo y nodos adyacentes. La Práctica No. 04 extiende el programa con los algoritmos de Dijkstra y Bellman-Ford, la visualización de la ruta más corta resaltada sobre el grafo, el costo total del camino y un análisis de eficiencia con distintas estructuras de datos.

Los objetivos del trabajo son:

- Representar grafos mediante matrices de adyacencia con validación de integridad de los datos.
- Implementar Dijkstra (con dos estructuras de datos) y Bellman-Ford para rutas más cortas.
- Visualizar el grafo y resaltar la ruta más corta, mostrando además el costo total.
- Comparar experimentalmente la eficiencia de los algoritmos y contrastarla con su complejidad teórica.

El resto del documento se organiza así: la Sección II resume el marco teórico, la Sección III describe el desarrollo del software, la Sección IV presenta los resultados, la Sección V discute limitaciones y la Sección VI concluye.

II. MARCO TEÓRICO

A. Grafos dirigidos y no dirigidos

Un grafo está compuesto por un conjunto no vacío de vértices V y un conjunto de aristas A , donde cada arista conecta dos vértices. En un grafo no dirigido las aristas no tienen orden, es decir, $(v, w) = (w, v)$. En un grafo dirigido (dígrafo) las aristas son pares ordenados $\langle v, w \rangle \neq \langle w, v \rangle$, donde v es la cola y w es la cabeza de la arista [3].

B. Terminología

- **Nodos adyacentes:** nodos unidos a v mediante una arista. En grafos dirigidos se distingue entre adyacentes de salida (w con $\langle v, w \rangle \in A$) y de entrada (u con $\langle u, v \rangle \in A$).
- **Camino:** secuencia de vértices tal que cada par consecutivo está unido por una arista. Su longitud es el número de aristas (número de nodos $- 1$).
- **Ciclo:** camino cuyo primer y último vértice son iguales; en grafos no dirigidos, las aristas deben ser distintas.
- **Grafo conexo:** existe un camino entre cualquier par de vértices (fuertemente conexo si es dirigido).

C. Representación mediante matriz de adyacencia

La matriz de adyacencia M es una matriz cuadrada de tamaño $n \times n$ donde $M[i][j]$ indica la existencia de una arista de i hacia j . En grafos ponderados, $M[i][j]$ almacena el peso de la arista. En grafos no dirigidos la matriz es simétrica. Su costo de memoria es $O(V^2)$. La alternativa es la lista de adyacencia, que guarda para cada nodo únicamente sus vecinos y requiere $O(V + E)$ de memoria.

D. Algoritmo de Dijkstra

Dijkstra calcula la ruta más corta desde un nodo origen hacia los demás nodos en un grafo con pesos no negativos [1]. Sus pasos son: (1) inicializar la distancia del origen en 0 y la de los demás nodos en infinito; (2) elegir el nodo no visitado con menor distancia; (3) actualizar las distancias de sus vecinos si se encuentra un camino más corto (relajación); (4) repetir hasta visitar todos los nodos. Con una búsqueda lineal del mínimo su complejidad es $O(V^2)$; con una cola de prioridad binaria y lista de adyacencia es $O((V + E) \log V)$.

E. Algoritmo de Bellman-Ford

Bellman-Ford también calcula rutas más cortas desde un origen, pero admite pesos negativos [1]. Inicializa las distancias igual que Dijkstra, relaja todas las aristas $|V| - 1$ veces y realiza una iteración adicional para comprobar la existencia de ciclos de peso negativo: si todavía puede mejorar alguna distancia, existe un ciclo de peso negativo alcanzable y la ruta más corta no está definida. Su complejidad es $O(V \cdot E)$, que se convierte en $O(V^3)$ cuando el grafo se recorre sobre una matriz de adyacencia.

TABLA I. COMPLEJIDAD TEÓRICA DE LAS VARIANTES IMPLEMENTADAS

Algoritmo	Estructura	Complejidad	Pesos negativos
Dijkstra (búsqueda lineal del mínimo)	Matriz de adyacencia	$O(V^2)$	No
Dijkstra (heap)	Lista de adyacencia	$O((V + E) \log V)$	No
Bellman-Ford	Matriz de adyacencia	$O(V^3)$	Sí

III. METODOLOGÍA Y DESARROLLO DEL SOFTWARE

A. Herramientas y arquitectura

La aplicación se desarrolló en Python 3 con las bibliotecas estándar `math`, `time`, `heapq`, `io`, `contextlib` y `tkinter`, además de `networkx` y `matplotlib` para la representación gráfica. El código se separó en dos módulos:

- **mainGrafoPart2.py:** lógica del problema (validación, representación matemática y gráfica, adyacencia, ciclos, algoritmos de ruta más corta y comparación de eficiencia). Incluye además una versión por consola (`main` y `menuConceptos`).
- **interfazGrafo.py:** interfaz gráfica en Tkinter (clase `AppGrafo`) que importa las funciones del módulo anterior y captura su salida de consola para mostrarla en un área de texto.

Esta separación permite reutilizar la misma lógica desde la consola y desde la interfaz, y fue la forma de dar continuidad a la práctica anterior, como lo permite la guía de la Práctica No. 04.

B. Ingreso de datos y validación de integridad

El usuario indica si el grafo es dirigido, si es ponderado y el número de nodos n . Luego ingresa la matriz de adyacencia de $n \times n$ valores, ya sea por consola (una fila por línea) o en una cuadrícula de campos de texto en la interfaz. Se implementaron las siguientes validaciones:

- El número de nodos debe ser un entero mayor que 0.
- En consola, cada fila debe contener exactamente n valores.
- En grafos ponderados, cada valor se convierte con la función `convertirAPeso`, que rechaza texto no numérico y los valores `nan` e `inf`.
- En grafos sin peso solo se aceptan 0, 1 o T (verdadero), convertidos a 0 o 1, siguiendo la notación de la guía.
- En grafos no dirigidos, la función `validarMatrizSimetrica` comprueba que $M[i][j] = M[j][i]$; si no se cumple, se

informa la posición del error y se solicita de nuevo la matriz.

- La función `avisarSiHayLazos` advierte cuando existen auto-conexiones ($M[i][i] \neq 0$).
- Los nodos de origen, destino o consulta deben estar entre 1 y n .

C. Representación matemática y gráfica

La función `mostrarRepresentacionMatematica` construye el conjunto $V = \{v_1, \dots, v_n\}$ y el conjunto de aristas E . En grafos dirigidos las aristas se escriben como pares ordenados (v_i, v_j) y en no dirigidos como conjuntos $\{v_i, v_j\}$; en grafos ponderados se agrega el peso como tercer elemento. Para evitar duplicados en grafos no dirigidos solo se recorre el triángulo superior de la matriz.

La función `mostrarRepresentacionGrafica` crea un `DiGraph` o un `Graph` de `networkx` según el tipo de grafo, calcula las posiciones con `spring_layout` (semilla fija para obtener siempre el mismo dibujo) y lo dibuja con `matplotlib`, mostrando flechas en grafos dirigidos y etiquetas de peso en grafos ponderados. Si se recibe una ruta en el parámetro `caminoResaltar`, los nodos de la ruta se pintan de naranja y sus aristas de rojo con mayor grosor, cumpliendo el requisito de resaltar la ruta más corta.

D. Nodos adyacentes, caminos y ciclos

La función `obtenerNodosAdyacentes` devuelve, para un nodo dado, los nodos con arista de salida y, en grafos dirigidos, también los de entrada. Como evidencia del concepto de camino, el programa muestra la ruta encontrada entre dos nodos y su costo mediante `calcularCostoCamino`, que suma los pesos de las aristas consecutivas.

Para evidenciar el concepto de ciclo se implementaron dos búsquedas en profundidad (DFS):

- **detectarCicloDirigido:** usa tres colores por nodo (0 = no visitado, 1 = en la pila de recursión, 2 = terminado). Si desde un nodo u se alcanza un nodo v de color 1, existe una arista de retorno y se reconstruye el ciclo siguiendo los predecesores.
- **detectarCicloNoDirigido:** usa un arreglo de visitados y el padre de cada nodo; un vecino ya visitado que no sea el padre indica un ciclo.

Ambas funciones devuelven un único ciclo de ejemplo (o `None` si el grafo es acíclico), no la lista completa de ciclos.

E. Dijkstra con búsqueda lineal del mínimo

La función `encontrarCaminoDijkstra` mantiene un arreglo de distancias, un arreglo de nodos anteriores y una lista de nodos pendientes. En cada iteración se extrae con `min()` el nodo pendiente de menor distancia y se relajan sus aristas. El proceso termina anticipadamente al extraer el nodo destino o cuando el mínimo tiene distancia infinita. El camino se reconstruye recorriendo los nodos anteriores desde el destino. Su complejidad es $O(V^2)$.

```
while colaPrioridad:
    nodoMenorDistancia=min(colaPrioridad, key=lambda v:
distancia[v])
    colaPrioridad.remove(nodoMenorDistancia)
```

```

    if distancia[nodoMenorDistancia]==float("inf") or
nodoMenorDistancia==destino:
        break
    for j in range(n):
        if matriz[nodoMenorDistancia][j]!=0 and j in
colaPrioridad:

nuevaDist=distancia[nodoMenorDistancia]+matriz[nodoMenorDist
ancia][j]

    if nuevaDist<distancia[j]:
        distancia[j]=nuevaDist
        nodoAnterior[j]=nodoMenorDistancia

```

F. Dijkstra con lista de adyacencia y heap

Para analizar el efecto de las estructuras de datos, la función `encontrarCaminoDijkstraHeap` primero convierte la matriz en una lista de adyacencia (`construirListaAdyacencia`) y luego usa una cola de prioridad con `heapq`, que almacena tuplas (distancia, nodo). Las entradas desactualizadas se descartan al extraerlas comparando la distancia con la mejor conocida. La parte iterativa tiene complejidad $O((V + E) \log V)$; sin embargo, como el grafo se recibe como matriz, la construcción de la lista cuesta $O(V^2)$ en cada llamada.

G. Bellman-Ford

La función `encontrarCaminoBellman` relaja todas las aristas con tres bucles anidados ($V - 1$ pasadas $\times V \times V$ celdas de la matriz), por lo que su complejidad es $O(V^3)$. Después de las $V - 1$ pasadas realiza una pasada adicional: si todavía puede reducir alguna distancia, informa que existe un ciclo de peso negativo y devuelve `None`. También devuelve `None` si el destino es inalcanzable.

```

for _ in range(n - 1):
    for i in range(n):
        for j in range(n):
            if matriz[i][j] != 0:
                pesoArista = matriz[i][j]
                if costo[i] != float('inf') and
(costo[i] + pesoArista) < costo[j]:
                    costo[j] = costo[i] + pesoArista
                    predecesor[j] = i

```

H. Selección del algoritmo y manejo de pesos negativos

La función `tienePesosNegativos` revisa la matriz antes de aplicar Dijkstra. Si el usuario elige Dijkstra (en cualquiera de sus variantes) y existen pesos negativos, el programa lo informa y resuelve automáticamente con Bellman-Ford, ya que Dijkstra no garantiza resultados correctos con pesos negativos.

I. Interfaz gráfica

La interfaz (clase `AppGrafo`, Tkinter) ofrece casillas para indicar si el grafo es dirigido o ponderado, botones de opción para elegir el algoritmo (Dijkstra, Dijkstra con heap o Bellman-Ford), un campo para el número de nodos y un botón que genera la cuadrícula de entrada de la matriz. Los botones inferiores permiten procesar el grafo, ver la gráfica, consultar nodos adyacentes, buscar un camino, detectar ciclos y comparar la eficiencia. Los resultados se muestran en un área de texto con desplazamiento; para ello se redirige la salida estándar con `contextlib.redirect_stdout` hacia un objeto `io.StringIO`, de modo que las funciones de la lógica se reutilizan sin modificarlas.

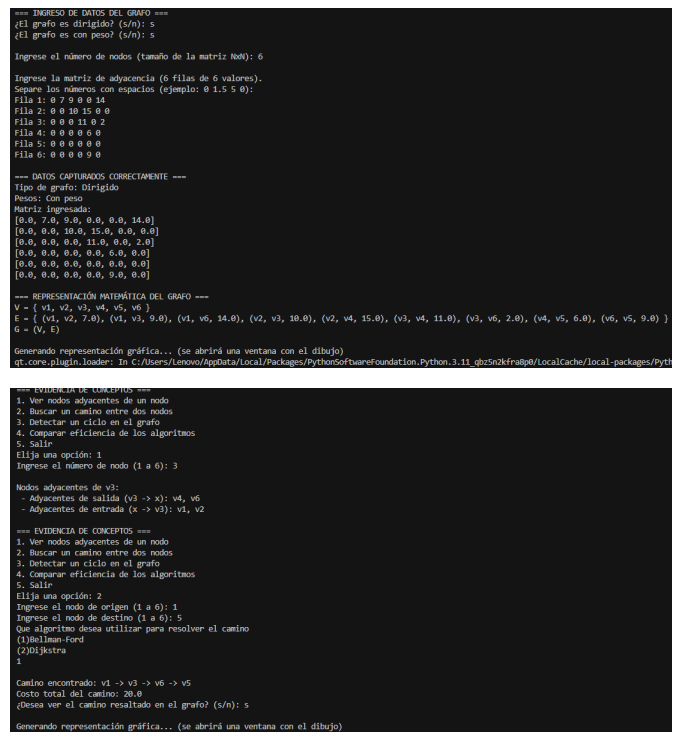


Fig. 1. Captura de la interfaz principal con una matriz cargada y el resultado de Procesar grafo

J. Comparación de eficiencia

La función `compararEficiencia` ejecuta cada algoritmo aplicable un número de repeticiones (1000 por defecto) con `time.perf_counter`, calcula el tiempo promedio por ejecución en milisegundos y verifica que el costo de la ruta sea el mismo. Si hay pesos negativos solo se mide Bellman-Ford. Al final imprime la complejidad teórica de cada variante.

IV. RESULTADOS

A. Grafo de prueba

Se usó un grafo dirigido y ponderado de seis nodos, con la siguiente matriz de adyacencia (0 indica ausencia de arista):

	v1	v2	v3	v4	v5	v6
v1	0	7	9	0	0	14
v2	0	0	10	15	0	0
v3	0	0	0	11	0	2
v4	0	0	0	0	6	0
v5	0	0	0	0	0	0
v6	0	0	0	0	9	0

La representación matemática generada por el programa fue:

$$V = \{ v1, v2, v3, v4, v5, v6 \}$$

$$E = \{ (v1, v2, 7.0), (v1, v3, 9.0), (v1, v6, 14.0), (v2, v3, 10.0), (v2, v4, 15.0), (v3, v4, 11.0), (v3, v6, 2.0), (v4, v5, 6.0), (v6, v5, 9.0) \}$$

$$G = (V, E)$$

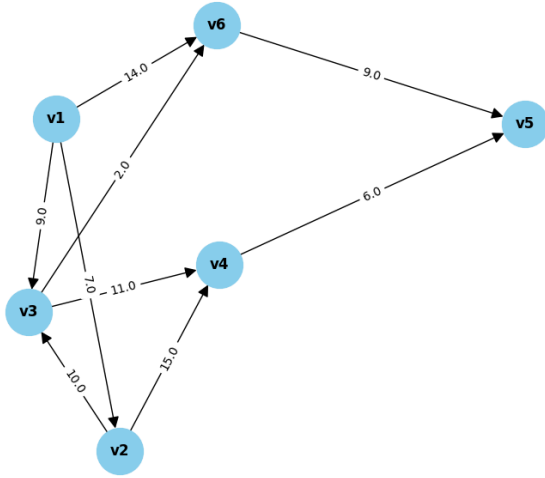


Fig. 2. Representación gráfica del grafo de prueba

B. Nodos adyacentes

Para el nodo v3 el programa reportó como adyacentes de salida v4 y v6, y como adyacentes de entrada v1 y v2, lo cual coincide con las aristas (v3, v4), (v3, v6), (v1, v3) y (v2, v3) del conjunto E.

C. Ruta más corta

Para origen v1 y destino v5, las tres variantes (Dijkstra, Dijkstra con heap y Bellman-Ford) encontraron el mismo resultado:

Camino encontrado: v1 → v3 → v6 → v5

Costo total del camino: 20.0

El costo se verifica manualmente: 9 (v1→v3) + 2 (v3→v6) + 9 (v6→v5) = 20, menor que las alternativas v1→v2→v3→v6→v5 (28) y v1→v6→v5 (23).

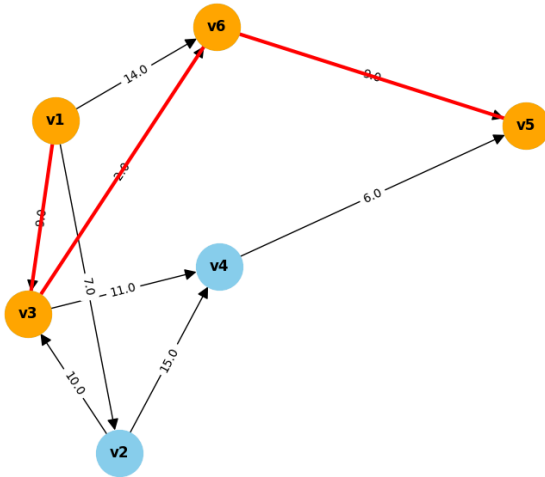


Fig. 3. Captura del grafo con la ruta v1 → v3 → v6 → v5 resaltada

D. Detección de ciclos

El grafo de prueba es acíclico, por lo que detectarCicloDirigido devolvió None y la interfaz mostró que no se encontró ningún ciclo. Al agregar la arista v5 → v1 con

peso 3, el programa detectó el ciclo v1 → v2 → v3 → v4 → v5 → v1. En un grafo no dirigido de cuatro nodos con aristas {v1,v2}, {v1,v3}, {v2,v3} y {v3,v4}, la detección arrojó el ciclo v1 → v2 → v3 → v1.

E. Pesos negativos y ciclos de peso negativo

En un grafo dirigido con aristas v1→v2 (4), v1→v3 (5), v2→v3 (-3) y v3→v4 (2), Bellman-Ford encontró la ruta v1 → v2 → v3 → v4 con costo 3, que es menor que el de la ruta directa v1 → v3 → v4 (7). En un grafo con el ciclo v1 → v2 → v3 → v1 de peso total -1 (aristas de pesos 1, -3 y 1), el programa mostró el aviso "se detectó un ciclo de peso negativo" y no devolvió ruta.

F. Validaciones

Al ingresar en un grafo no dirigido una matriz con M[1][2] = 1 y M[2][1] = 0, el programa informó que la matriz no es simétrica en esa posición. Los valores no numéricos, nan, inf o distintos de 0, 1 y T (en grafos sin peso) fueron rechazados con mensajes de error, y los nodos fuera del rango 1 a n fueron rechazados al consultar adyacentes o rutas.

G. Eficiencia

Sobre el grafo de seis nodos, con 1000 repeticiones, los tiempos promedio por ejecución (v1 → v5) fueron:

TABLA II. TIEMPO PROMEDIO POR EJECUCIÓN EN EL GRAFO DE PRUEBA (n = 6)

Algoritmo	Tiempo (ms)	Costo
Bellman-Ford (matriz)	0.01349	20
Dijkstra (lista y min)	0.00633	20
Dijkstra (heap)	0.00411	20

Para observar la tendencia con tamaños mayores se generaron grafos dirigidos aleatorios (semilla fija, densidad aproximada del 10 %, pesos enteros entre 1 y 20) y se midió la ruta de v1 a vn:

TABLA III. TIEMPO PROMEDIO (MS) SEGÚN EL NÚMERO DE NODOS

n	Bellman-Ford	Dijkstra (lista y min)	Dijkstra (heap)
20	0.265	0.029	0.020
50	3.763	0.134	0.079
100	30.010	0.087	0.321
200	253.347	3.618	2.607

Bellman-Ford pasó de 30.0 ms a 253.3 ms al duplicar n de 100 a 200 (factor ≈ 8.4), lo que concuerda con el crecimiento cúbico esperado. Las variantes de Dijkstra crecen mucho más lentamente. Los valores de Dijkstra en n = 100 son menores a los de n = 50 porque ambos algoritmos terminan al extraer el destino y, según el grafo aleatorio, esto puede ocurrir muy pronto; por eso estas mediciones son indicativas y varían con el grafo y el equipo. [Nota: los tiempos de las Tablas II y III se midieron en el entorno de desarrollo del informe; se recomienda repetirlos en el computador del autor con el botón "Comparar eficiencia" y reemplazar los valores.]

V. DISCUSIÓN Y LIMITACIONES

- **Convención del cero:** al usar 0 como ausencia de arista, no es posible representar aristas de peso 0. Si se

requirieran, habría que usar otro valor centinela (por ejemplo, infinito o None).

- **Pesos negativos en grafos no dirigidos:** como una arista no dirigida con peso negativo equivale a dos aristas dirigidas negativas, forma un ciclo de peso negativo ($v \rightarrow w \rightarrow v$); Bellman-Ford lo reportará, lo cual es el comportamiento teóricamente correcto.
- **Costo de construir la lista de adyacencia:** la variante con heap recibe una matriz, por lo que construir la lista cuesta $O(V^2)$ en cada llamada; la ventaja de $O((V + E) \log V)$ se aprovecha plenamente cuando el grafo ya se almacena como lista, sobre todo en grafos dispersos.
- **Parada anticipada:** ambas variantes de Dijkstra se detienen al alcanzar el destino, lo cual es correcto para una consulta origen-destino, pero hace que sus tiempos dependan de la posición del destino. Bellman-Ford siempre realiza las $V - 1$ pasadas, aunque podría detenerse si una pasada no produce cambios.
- **Ciclos:** los detectores devuelven un solo ciclo, no todos los existentes.
- **Visualización:** `spring_layout` con semilla fija mantiene la posición de los nodos entre dibujos, pero en grafos densos las etiquetas de peso pueden solaparse.

VI. CONCLUSIONES

- Se desarrolló una aplicación en Python, con versión de consola e interfaz Tkinter, que cumple los requisitos de las Prácticas 03 y 04: ingreso de la matriz de adyacencia, validación de datos, representación matemática y gráfica, evidencias de nodos adyacentes, camino y ciclo, y cálculo de la ruta más corta con costo total.
- Las tres variantes de rutas más cortas obtuvieron el mismo camino y costo en las pruebas, lo que respalda su correcta implementación.
- Bellman-Ford es el único algoritmo implementado que maneja pesos negativos y detecta ciclos de peso negativo, a cambio de un mayor costo computacional ($O(V^3)$ con matriz).
- Dijkstra es más eficiente cuando todos los pesos son no negativos, y el uso de lista de adyacencia con cola de prioridad mejora el rendimiento, especialmente en grafos grandes y dispersos.
- Como trabajo futuro se propone almacenar el grafo directamente como lista de adyacencia, detener Bellman-Ford cuando no haya cambios, permitir aristas de peso cero y listar todos los ciclos.

REFERENCIAS

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein, *Introduction to Algorithms*, 3.^a ed. Cambridge, MA, EE. UU.: MIT Press, 2009.
- [2] R. Sedgewick y K. Wayne, *Algorithms*, 4.^a ed. Boston, MA, EE. UU.: Addison-Wesley, 2011.
- [3] R. Diestel, *Graph Theory*, 5.^a ed. Berlín, Alemania: Springer, 2017.
- [4] J. L. Gross y J. Yellen, *Graph Theory and Its Applications*, 2.^a ed. Boca Raton, FL, EE. UU.: CRC Press, 2005.
- [5] S. Even, *Graph Algorithms*, 2.^a ed. Cambridge, Reino Unido: Cambridge University Press, 2011.
- [6] J. L. Gersting, *Mathematical Structures for Computer Science*, 7.^a ed. Nueva York, NY, EE. UU.: W. H. Freeman, 2014.
- [7] A. A. Hagberg, D. A. Schult y P. J. Swart, "Exploring network structure, dynamics, and function using NetworkX", en *Proc. 7th Python in Science Conf. (SciPy)*, 2008, pp. 11–15.
- [8] S. González Domínguez, *Implementación de los algoritmos de Dijkstra y Bellman-Ford para el cálculo de caminos mínimos en grafos*, Universidad Politécnica de Madrid, 2022.